

Déploiement d'une Infrastructure Cloud Hybride et Pipeline DevOps

---

Élaboré par :
Mr. EL BADRY Aymane
Mme. EL HABTI Oumaima

Sous la direction de : Pr. Benabbou

Année universitaire :2025 – 2026

Plan de la Présentation

01

Contexte et Problématique

02

Architecture Globale

03

Infrastructure as Code (Terraform)

04

Configuration et Optimisation (Ansible)

05

Conteneurisation et Orchestration (Docker & K3s)

06

Automatisation CI/CD (GitHub Actions)

07

Résultats et Conclusion

Contexte & Problématique

Objectif :

Mettre en place une chaîne DevOps complète de bout en bout.

Contrainte Majeure :

Utilisation de l'offre AWS Free Tier (t3.micro) limitée à **1 Go de RAM**.



Architecture Globale (Le Pipeline)

Explication visuelle du flux de travail.

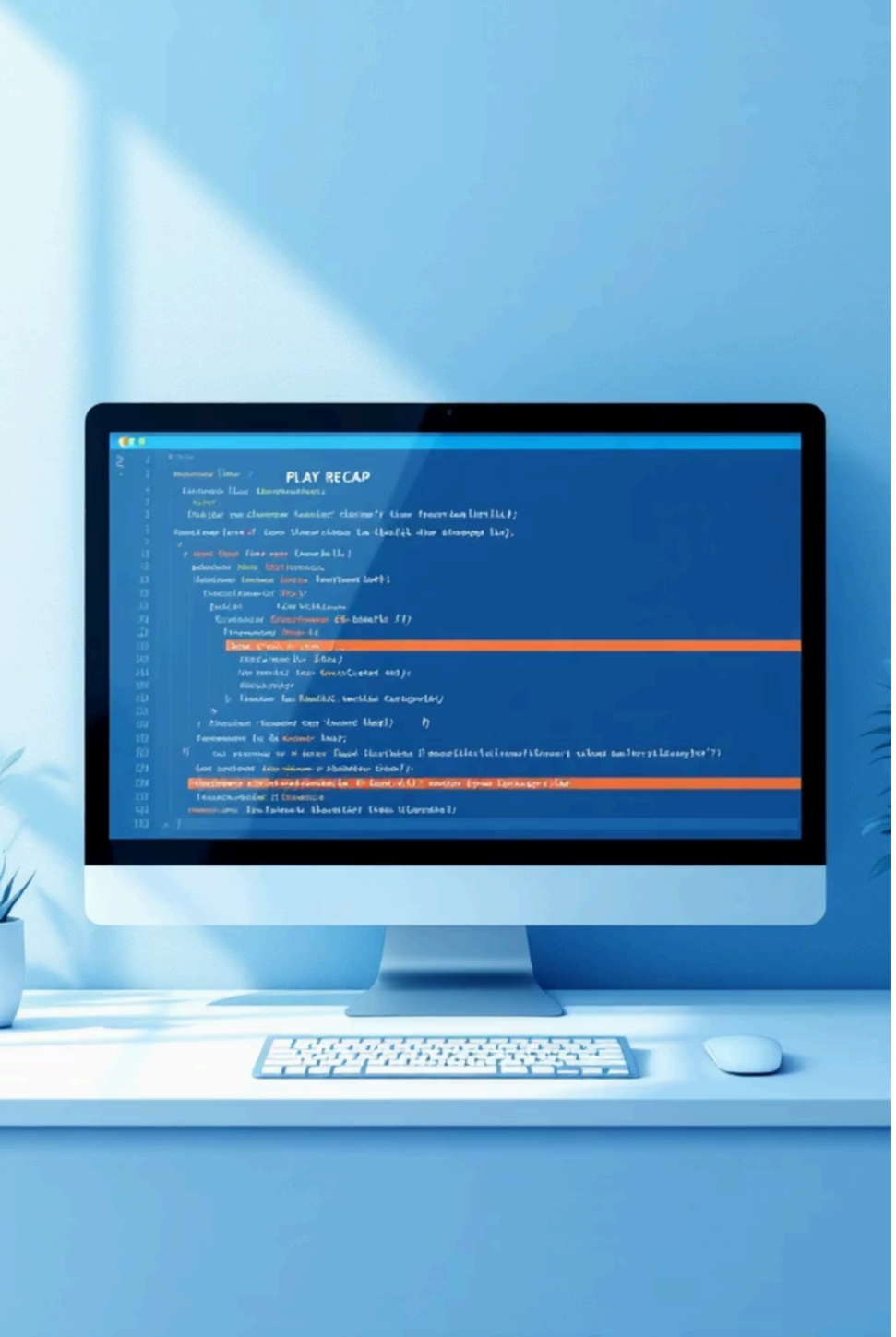


Infrastructure as Code (Terraform)

Automatisation de la création des ressources sur AWS (Master & Worker).

```
instance_type = "t3.micro"
```

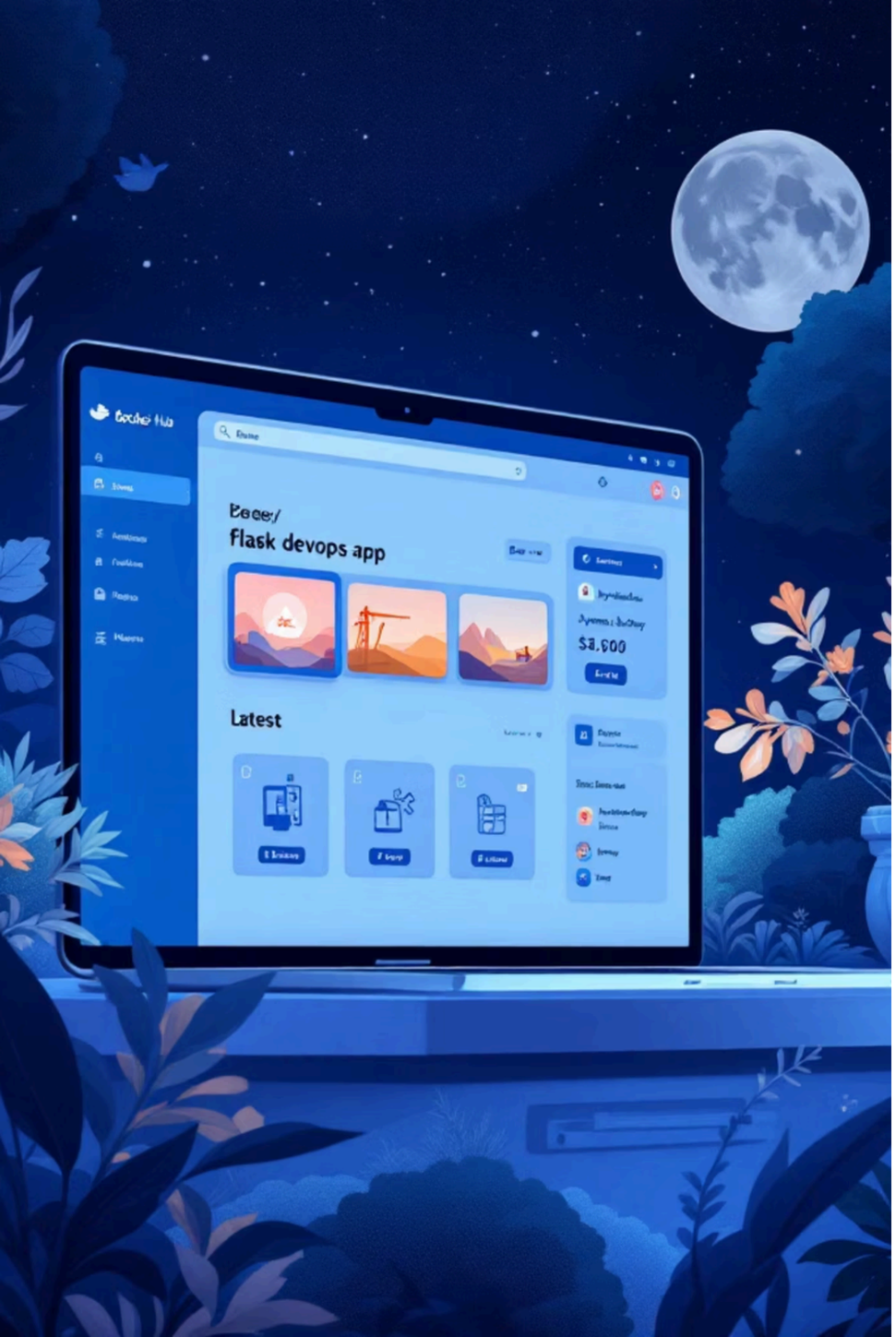




Configuration & Le "Secret" (Ansible)

Configuration Agentless et contournement de la limite de RAM.

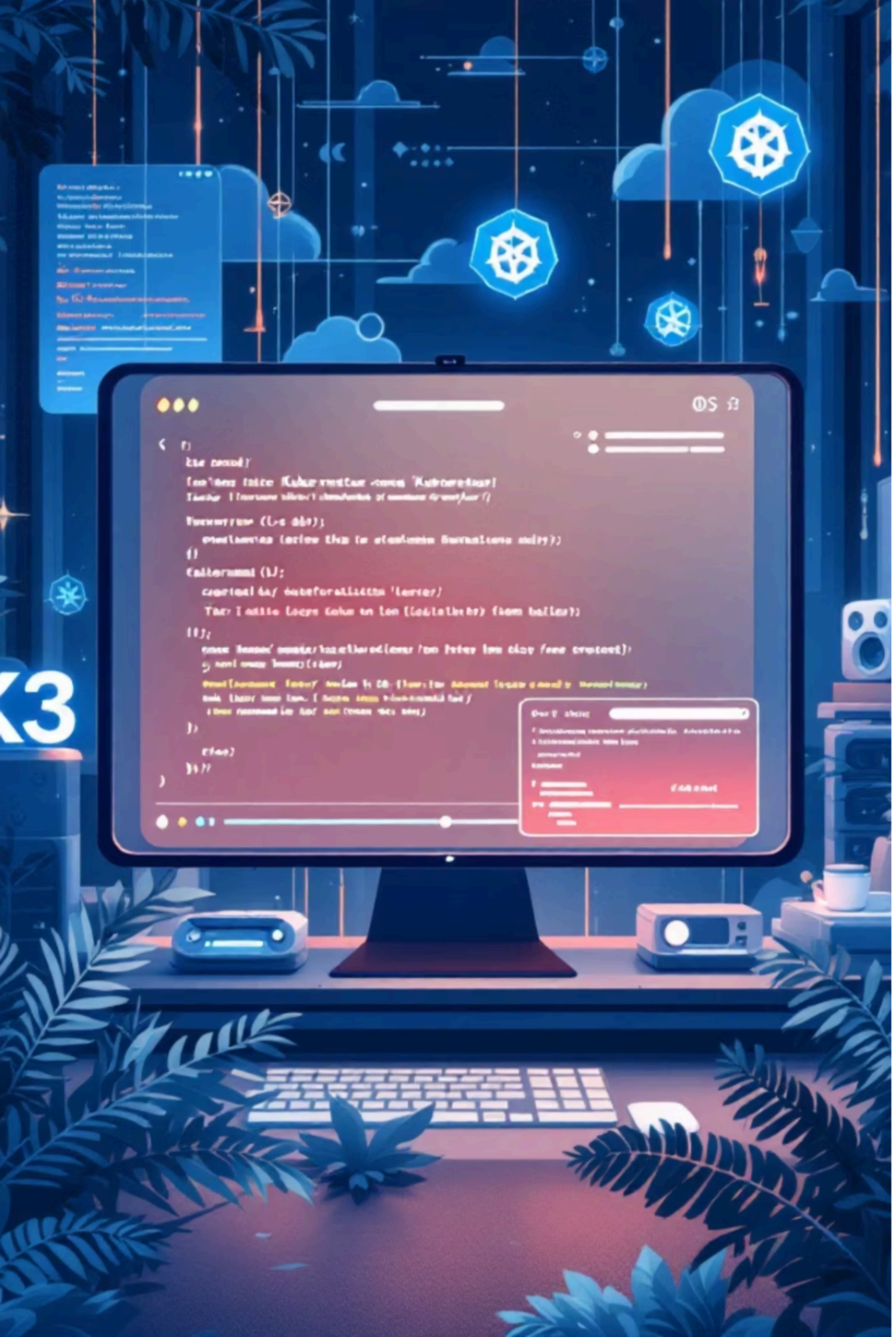
```
- name: Create swap file
 command: fallocate -l 1G /swapfile
 become: yes
```



Conteneurisation (Docker)

Encapsulation de l'API Flask et réduction de la taille de l'image.

```
FROM python:3.9-slim
COPY . .
```



L'Orchestration (Kubeadm vs K3s)

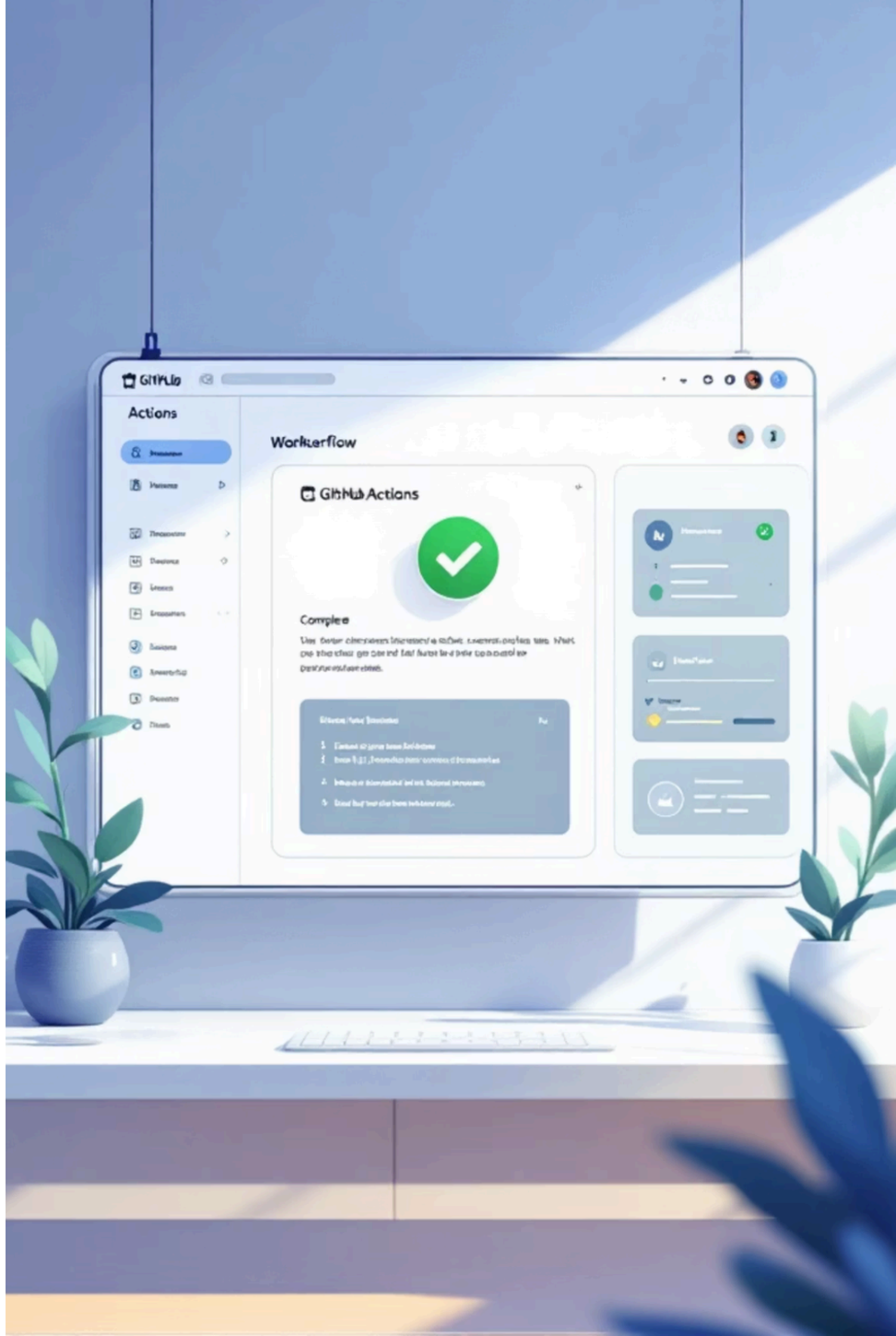
- **Échec initial :** Saturation mémoire avec Kubeadm.
- **Solution :** Migration vers **K3s** (Léger) et application de limites de RAM (128Mi) au Pod.

```
resources:
 limits:
 memory: "128Mi"
```

Automatisation CI/CD (GitHub Actions)

Suppression des interventions manuelles et "Zero Downtime Deployment".

```
sudo kubectl rollout restart deployment/flask-app
```



Résultat Final en Production

L'API Flask est accessible publiquement de manière stable et sécurisée.



Merci pour votre attention !

